LeetCode 每日一题 (2026年05月12日)

今日主题：双指针 (Two Pointers) — 两数之和II & 滑动窗口

【第2天】双指针 — Two Sum II & 滑动窗口

一、算法核心思想

双指针利用两个指针协同移动来遍历数据，将 $O(n^2)$ 降到 O(n)。常见模式：

- 对撞指针：两端向中间移动（有序数组求和）
- 滑动窗口：同向移动维护区间（子串/子数组问题）
- 快慢指针：不同速度遍历（链表环检测）

二、经典题目：167. Two Sum II — Input Array Is Sorted

题目描述

在一个升序排列的数组中，找到两个数使它们的和等于 target。已知有且只有一对解，返回两个数的下标（1-based）。

输入：`numbers = [2,7,11,15], target = 9`
输出：`[1,2]`

解法：对撞指针

利用数组有序性：如果两数和太小，左指针右移增大和；两数和太大，右指针左移减小和。

```
// 时间复杂度 O(n)，空间复杂度 O(1)
vector<int> twoSum(vector<int>& numbers, int target) {
    int left = 0, right = numbers.size() - 1;

    while (left < right) {
        int sum = numbers[left] + numbers[right];

        if (sum == target) {
            // 题目要求返回 1-based 下标
            return {left + 1, right + 1};
        } else if (sum < target) {
            left++;    // 和太小 → 左指针右移，增大 sum
        } else {
            right--;   // 和太大 → 右指针左移，减小 sum
        }
    }
    return {};  // 题目保证有解，此行不会执行
}
```

正确性证明：假设最优解是 i 和 j。当 left < i 时，sum 必然小于 target，所以 left 一直右移直到到达 i。同理 right 最终到达 j。

三、经典题目：76. Minimum Window Substring（最小覆盖子串）

题目描述

给定字符串 s 和 t，找出 s 中包含 t 所有字符的最短子串。

输入：s = "ADOBECODEBANC", t = "ABC"
输出："BANC"

解法：滑动窗口

```
// 时间复杂度 O(n)，空间复杂度 O(1)（字符集大小固定）
string minWindow(string s, string t) {
    // 1. 统计 t 中每个字符需要的数量
    vector<int> need(128, 0);
    for (char c : t) need[c]++;

    int left = 0, right = 0;        // 滑动窗口 [left, right)
    int count = t.size();           // 还需要匹配的字符总数
    int min_start = 0, min_len = INT_MAX;

    // 2. 右指针不断向右扩展窗口
    while (right < s.size()) {
        char c = s[right];
        right++;

        // 如果当前字符是需要的，减少需求计数
        if (need[c] > 0) count--;
        need[c]--;  // 注意：即使不需要也会减成负数

        // 3. 当窗口包含所有字符时，尝试收缩左边界
        while (count == 0) {
            // 更新最短子串
            if (right - left < min_len) {
                min_start = left;
                min_len = right - left;
            }

            // 左边界字符移出窗口
            char d = s[left];
            left++;
            need[d]++;
            // 如果移出的是必须字符，需求计数增加
            if (need[d] > 0) count++;
        }
    }
    return min_len == INT_MAX ? "" : s.substr(min_start, min_len);
}
```

解法对比

方法	时间复杂度	说明
暴力枚举	$O(n^3)$	枚举所有子串，检查是否覆盖
滑动窗口	$O(n)$	每个字符最多被左右指针各访问一次

四、触类旁通

- **3.** 无重复字符的最长子串 — 滑动窗口经典题
- **142.** 环形链表 II — 快慢指针（Floyd 判圈法）
- **88.** 合并两个有序数组 — 逆向双指针从后往前填充